

## SYLLABUS CONTENT

By the end of this chapter, you should be able to:

- ▶ A3.4.1 Outline the different types of databases as approaches to storing data
- ▶ A3.4.2 Explain the primary objectives of data warehouses in data management and business intelligence
- ▶ A3.4.3 Explain the role of online analytical processing (OLAP) and data mining for business
- ▶ A3.4.4 Describe the features of distributed databases

## A3.4.1 Types of databases as approaches to storing data

Database models represent frameworks that determine the logical structure of a database and influence how data is stored, organized and manipulated. Different database models cater to different types of applications and requirements.

### ■ NoSQL databases

◆ **NoSQL database:** a database designed to handle large volumes of data and diverse data types, structured differently from relational databases.

**NoSQL databases** are designed to handle large volumes of data and diverse data types. They offer flexibility and scalability, making them suitable for modern web applications, such as e-commerce platforms, big data and real-time analytics.

NoSQL databases store data differently from relational databases. There are four main types of NoSQL databases: document databases, key-value databases, wide-column store databases and graph databases.

#### Document databases

In document databases, data is stored in documents like JSON (JavaScript Object Notation) objects. The documents contain pairs of fields and values, and document databases are used for content management systems and e-commerce platforms. They offer a flexible data model, suitable for semi-structured and unstructured data sets. They provide an easy way to represent hierarchical data, but a disadvantage is that they pose a risk of data redundancy and inconsistency. An example of such a database is MongoDB. Here is an example:

```
{
  "_id": "12345",
  "name": "blabla",
  "email": "blabla@car.com",
  "address": {
    "street": "bloblo street",
    "city": "omega city",
  },
  "services": ["transport", "tourism"]
}
```

## Key-value databases

In key-value databases, data is stored as key-value pairs. Key-value databases are used for real-time analytics, caching and session management. They are simple databases that allow for fast read and write operations; however, they have limited querying capabilities. An example of a key-value database is Redis. Here is an example:

Key: user:12345

Value: {"name": "blabla", "email": "blabla@car.com", "job": "transporter"}

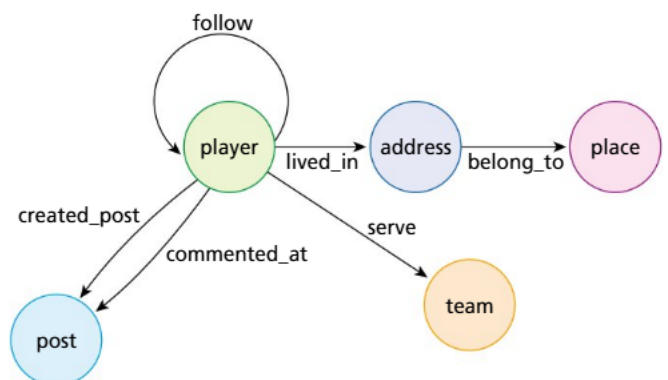
## Wide-column store databases

In wide-column store databases, data is stored in tables, rows and dynamic columns. Different rows can have different sets of columns. They enable efficient retrieval of sparse and wide data and are used in big-data applications or real-time analytics. They are proven to be efficient for read and write operations on large data sets and are easy to scale horizontally; however, this is a complex model to implement. An example of a wide-column store database is Cassandra. Here is an example:

| name   | id    | email          | dateOfBirth |
|--------|-------|----------------|-------------|
| blabla | 12345 | blabla@car.com |             |
| nathan | 1234  |                | 12-12-2000  |

## Graph databases

In graph databases, data is stored as nodes and edges in a graph structure. Nodes usually store data about people, places or things, and edges store data about the relationships between nodes. They are used for social-media platforms or recommendation engines. Graph databases are great for representing and querying complex relationships, but they use a specialized query language, and it is more complex to maintain and optimize them. An example of a graph database is OrientDB. Here are two examples of graph databases:



◆ **Cloud database:**

a database that runs on cloud computing platforms, providing scalability, high availability and flexible resource management.

◆ **Spatial database:**

a database optimized to store and query data related to objects in space, including points, lines and polygons.

◆ **In-memory**

**database:** a database that stores data entirely in the main memory (RAM) rather than on disk, providing extremely fast read and write operations.

## ■ Cloud databases

**Cloud databases** are databases that run on cloud computing platforms, providing scalability, high availability and flexible resource management. They can be NoSQL or SQL databases.

In cloud databases, pricing is based on the use of system resources, which can be provisioned on demand as needed to meet processing workloads. Organizations can choose between two models when they opt for cloud databases:

- **Self-managed database:** An infrastructure as a service (IaaS) environment, in which the database runs on a virtual machine on a system operated by a cloud provider. The provider manages and supports the cloud infrastructure, including servers, operating systems and storage devices. But the organization is responsible for database deployment, administration and maintenance, so it has full control over the database.
- **Managed database services:** Fully managed by the vendor, both the system infrastructure and the database platform are managed for the customer; the vendor handles provisioning, backups, scaling, patching, upgrades and other basic database administration functions; organizations monitor the database, and they can collaborate with the vendor on some administrative functions.

Examples of database-as-a-service (DBaaS) are Amazon DynamoDB and Azure Cosmos DB. They are used for startups, IoT or web-scale applications; they are fully managed, offer pay-as-you-go pricing and the organization has limited control over the infrastructure.

Examples of managed databases are Amazon RDS, Google Cloud SQL and Azure SQL database. They are used for web and enterprise applications, and they depend on cloud providers and potentially have higher costs.

## ■ Spatial databases

**Spatial databases** are optimized to store and query data related to objects in space, including points, lines, polygons, 3D shapes and coordinates. They support spatial data types and spatial indexes to access the data, and they support geometric functions. They are used for Geographic Information Systems (GIS), location-based services and mapping applications. Examples are: Oracle Spatial, PostGis and MongoDB with Geospatial Indexes. Such models efficiently store and query spatial data, but they use complex data types and queries that require specialized knowledge or experts.

## ■ In-memory databases

**In-memory databases** store data entirely in the main memory (RAM) rather than on disk, providing extremely fast read and write operations. They are used for real-time analytics, caching and gaming applications. They allow for extremely fast data access and transaction processing, but they have low latency for read and write operations. Examples are Redis and Oracle TimesTen.

## A3.4.2 Primary objectives of data warehouses in data management and business intelligence

◆ **Data warehouse:**

a specialized type of database designed for analytical purposes rather than transactional processing.

A **data warehouse** is a specialized type of database designed for analytical purposes rather than transactional processing. It is used to store and analyse large volumes of historical data from various sources to support decision-making processes within organizations. A data warehouse represents a repository of stored data related to a specific subject. It includes tools to extract, transform and load data into the repository and tools to manage and retrieve the metadata.

The characteristics of data warehouses are discussed below:

### ■ Append-only

This characteristic means that, when data is loaded using append-only, existing records are not updated, but instead are appended to tables as new rows. Therefore, at a later stage, the tables will contain different versions of the records, so that how they changed over time can be analysed.

### ■ Subject-oriented

Data warehouses are organized around key subjects or themes relevant to the organization, such as sales, marketing or finance. They help with organizing and presenting data in a way that is aligned with the analytical needs and objectives of the organization.

### ■ Integrated data

Data from multiple operational systems and external sources are integrated into a single repository using ETL (**extract, transform, load**) processes. This consolidates information from multiple sources into a centralized repository, providing a single source of truth for the organization. When ETL is carried out, certain precautions should be taken, such as:

- ensuring the extraction does not affect the source system
- ensuring the extracted data can be read by the current system
- ensuring the different data formats being extracted can be converted to become readable by the system and can be formatted
- ensuring that the data is relevant to what the user wants to extract and utilize.

### ■ Time-variant

Data warehouses store historical data to support time-based analysis, enabling comparisons and trend analysis over time. Data warehousing is time-dependent because the content in the data warehouse is only valid for a period, because the data undergoes changes dynamically, and its focus on change over time is time-variant.

### ■ Non-volatile

Data once loaded into the data warehouse is rarely updated or deleted, ensuring data integrity and consistency for analytical purposes.

### ■ Optimized for query performance

Data warehouses seek to determine the most efficient way to execute a given query by considering a variety of query execution strategies. It directly impacts the speed and efficiency of data retrieval and analysis processes.

## A3.4.3 The role of OLAP and data mining for business intelligence

**Business intelligence** refers to the technologies, applications and practices for collecting, integrating, analysing and presenting business information. Its aim is to support data-driven decision-making and improve business performance.

**Online analytical processing** (OLAP) and **data mining** are technologies used for data analysis and business intelligence, enabling organizations to extract valuable insights and make informed decisions from their data.

◆ **Extract:** to gather data from various operational databases, flat files, APIs, etc.

◆ **Transform:** to aggregate and transform data into a consistent format suitable for analysis.

◆ **Load:** to load transformed data into a data warehouse.

◆ **Business intelligence:** technologies, applications and practices for collecting, integrating, analysing and presenting business information.

◆ **Online analytical processing:** the software technology you can use to analyse business data from different points of view.

◆ **Data mining:** the process of sorting through large data sets to identify patterns and relationships that can help solve business problems through data analysis.



## ■ Role of OLAP in business intelligence

OLAP facilitates interactive analysis of multidimensional data, allowing users to explore data from various dimensions, such as time, product or region. It provides pre-aggregated views of data, which are optimized for querying and reporting, allowing for quick retrieval of summarized information, and supports decision-making processes. OLAP supports complex calculations and analytical functions directly on aggregated data, such as year-over-year comparisons. It supports data visualization techniques, such as charts, graphs and dashboards, and users can create ad hoc queries to explore data dynamically and answer specific business questions without needing to rely on predefined reports.

## ■ Role of data mining in business intelligence

Data mining involves discovering patterns and relationships within large data sets, using statistical algorithms, machine learning techniques and artificial intelligence to uncover hidden insights. It enables predictive modelling by analysing historical data to forecast future trends and outcomes, which helps with predicting customer behaviour, demand forecasting and risk assessment. As such, it helps in customer segmentation based on attributes and behaviours, by identifying customer segments with similar characteristics for targeted marketing campaigns and personalized customer experiences. It can detect anomalies in data, which may indicate fraud, errors or unusual patterns that require further investigation. Database segmentation can help to increase the profit of the organization, increase its reputation, increase the number of customers and provide better opportunities for growth.

Data mining techniques include the following:

- **Classification:**
  - A supervised learning technique that categorizes data into labels based on input features.
  - Used in spam email detection, sentiment analysis and credit scoring.
- **Clustering:**
  - An unsupervised learning technique that groups similar data points together into clusters based on their characteristics or proximity in feature space.
  - Finds patterns in customer behaviour by grouping and analysing variables to connect them; it can find previously unknown links that help in decision-making.
  - Used in market segmentation, customer profiling and anomaly detection.
- **Regression:**
  - Predicts continuous numerical values based on input variables, aiming to establish relationships between variables.
  - Used in sales forecasting, price predictions and risk assessment.
- **Association rule discovery:**
  - Identifies relationships or associations between items in large data sets, typically in transactional databases.
  - Looks at how entities or events are connected, and finds where one or more events may lead to another.
  - Correlates the presence of a set of items with another range of values for another set of variables, breaking up the data sets by variables such as location, age, gender.
  - Used in cross-selling recommendations.
- **Sequential pattern discovery:**
  - Discovers patterns or sequences in data, where events occur in a specific order, over a specific period of time (temporal patterns).
  - Used in web log analysis and clickstream analysis.

- **Anomaly detection:**

- Identifies rare or unusual patterns in data that do not conform to expected behaviour.
- Used in fraud detection, network security monitoring and equipment failure prediction.

### A3.4.4 Features of distributed databases

◆ **Distributed database:** a database made of two or more files located on different sites on the same network or on completely different networks.

A **distributed database** is a database made of two or more files located on different sites on the same network or on completely different networks. Although they are stored on different sites or different computer systems, they provide a fully functional, unified view of data to users and applications.

Distributed databases are used in different areas, such as online retailers using them to manage product catalogues, inventory and transactions across distributed warehouses, and telecom companies using them to manage subscriptions, network traffic analysis and service provisioning across different geographical locations.

Maintaining data consistency in a distributed system is crucial for ensuring the reliability, accuracy and trustworthiness of data across all nodes and users. This helps with reliable decision-making; avoiding data corruption; maintaining data integrity; and increasing user satisfaction and trust.

The role of atomicity, consistency, isolation and durability (ACID) to ensure reliable processing of transactions in distributed databases is as follows:

- **Atomicity**

Atomicity ensures that distributed transactions are either committed across all nodes or rolled back completely if any part of the transaction fails. This prevents partial updates and maintains data consistency across nodes.

- **Consistency**

Consistency in distributed systems ensures that all nodes have access to the same consistent view of data after a transaction is completed.

- **Isolation**

Isolation prevents interference between concurrent transactions executing on different nodes. It prevents the modification of the same data item by two different transactions.

- **Durability**

Durability ensures that committed transactions are reliably stored and replicated across distributed nodes.

The features of distributed databases are described below:

#### Concurrency control

Concurrency control refers to techniques used to manage simultaneous access and modifications to shared data across multiple nodes in a distributed database. It ensures that transactions execute correctly and maintain consistency, despite potential conflicts that may arise due to concurrent operations. Locking mechanisms can be used to enforce isolation and prevent conflicting operations, or unique timestamps can be assigned to each transaction to determine the order of execution. For example, different systems may attempt to access the same data at the same time, such as two systems attempting to update the same piece of data. If one starts the update and then the second finishes before the first is saved,

this could potentially lead to inconsistent updates. In such cases, the solution is to isolate the transactions; when one system is accessing the data, that transaction is locked, and it is released only after the transaction is committed.

### Data consistency

All copies of data across distributed nodes are synchronized and reflect the most recent, correct state of information. Strong consistency models ensure that all updates to data are visible to all nodes immediately after they occur.

### Data partitioning

Data partitioning improves performance, scalability and manageability by distributing data across multiple nodes or servers. It allows databases to handle large volumes of data and high transaction rates efficiently.

### Data security

By implementing robust security measures and continuously monitoring for threats and vulnerabilities, organizations can mitigate risks and safeguard sensitive data across distributed environments effectively. To achieve this, organizations can:

- encrypt sensitive data stored on disks or databases to protect against unauthorized access
- define roles and permissions that restrict access to data based on users' roles
- use strong authentication mechanisms (multifactor authentication) to verify user identities before granting access to the database.

### Distribution transparency

Distribution transparency refers to the ability to access and manipulate data across multiple nodes or servers in a transparent and seamless manner.

### Fault tolerance

Fault tolerance refers to systems' ability to continue operating and providing services even in the presence of hardware failures, software errors or network disruptions, by using data-redundancy systems, monitoring and recovery in case of failure detection.

### Global query processing

Global query processing involves the coordination and execution of queries that span multiple distributed nodes or databases.

### Location transparency

Location transparency refers to the ability of a system to hide the physical or logical location of resources and services (network address or server details) from users and applications. It ensures that users can access resources or services without being aware of their specific location, simplifying system management.

### Replication

Replication addresses the creation and maintenance of copies of data across multiple nodes, servers or locations, which enhances data availability, reliability and performance.

### Scalability

Scalability refers to the ability of the database system to handle increasing amounts of data and user requests by efficiently distributing workload across multiple nodes or servers. It ensures the database can grow to meet performance and capacity requirements as demands increase.

**ACTIVITY**

**Social skills:** Listen actively to other perspectives and ideas – there are different ways to solve a problem, some better than others; listen to advice and try new techniques and problem-solving strategies. In any collaborative environment, actively listening to the perspectives and ideas of others is crucial for effective problem-solving and decision-making. Different individuals bring diverse experiences, viewpoints and approaches to the table, each offering unique insights into how a problem can be addressed. By carefully listening to others, you not only gain a broader understanding of the problem, but you also become aware of innovative solutions that you might not have considered on your own.

**REVIEW QUESTIONS**

- 1 Define the term “data mining”.
- 2 Define the term “data warehouse”.
- 3 Describe how regression is used in data mining.
- 4 Describe the concurrency control feature in distributed databases. Explain how this can be achieved.
- 5 Explain the role of integrated data in a data warehouse.
- 6 Outline two methods to ensure the security of a data warehouse.
- 7 Compare classification and sequential pattern discovery in data mining.
- 8 Outline the differences between a database view and a data warehouse.

 **Linking questions**

- 1 What processes are needed to store data in database structures so that they can be used in machine learning? (A4)
- 2 How does database programming in SQL differ from programming computationally in a high-level language? (B2)
- 3 To what extent is the effectiveness of the distributed database determined by the network that connects the various tables? (A2)
- 4 How could machine learning be applied to databases? (A4)
- 5 How do programming languages interact with databases to store, retrieve and manipulate data? (B2)



## EXAM PRACTICE QUESTIONS

- 1 A telecommunications company is designing a relational database to store its desk tickets. The database will have the following tables:
- ```
Operator(OperatorID, OperatorName, Location)
Engineer(EngineerID, EngFirstName, EngLastName, EngLocation, Salary)
Ticket(TicketNo, Location, Status, OperatorID, TicketDate, TicketPriority)
Supplier(SupplierID, Email, PhoneNo)
Product(ProductID, ProductName, Price, SupplierID)
ProductOrder(TicketNo, OrderID, ProductID, Quantity, EngineerID)
```
- Identify the primary key for each of the tables described above. [3]
  - Identify three benefits of a relational database. [3]
  - Construct an ERD for the relational database. [3]
  - Construct the SQL DDL instructions to create the `Ticket` table. [1]
  - Construct the SQL DML statements to return the total product quantity ordered by the engineer with the engineer ID D893. [4]
  - Identify a foreign key in the `Product` table. [1]
  - State whether the database is normalized and whether it is in third normal form (3NF). [1]
  - Describe the characteristics of a database that is in third normal form (3NF). [3]
  - The following table is an example of the `Engineer` table.

| EngineerID | EngFirstName | EngLastName | EngLocation | Salary |
|------------|--------------|-------------|-------------|--------|
| D893       | Daniel       | Buchidau    | Trier       | 7000   |
| D894       | Constantin   | Constantin  | Heidelberg  | 6000   |
| D895       | Martin       | Bond        | Cologne     | 6500   |

- Define the term "tuple". Give an example of a tuple from the `Engineer` table. [2]
  - State the number of fields in the `Engineer` table. [1]
  - Construct an SQL statement to increase the salary of the engineer with the ID D894 by 300. [2]
  - Construct SQL statements to update the `Engineer` table to include two more fields: one called `Experience`, to store how many years of experience each engineer has, and one called `IncreaseDate`, which includes the date when the last salary raise occurred. [4]
  - Construct an SQL script to find the average salary in the `Engineer` table. [2]
- State what a transaction is and identify the four properties that define a transaction. [2]
  - Identify three advantages of using views in a database. [3]
  - Explain three data mining techniques. [6]
  - Compare cloud and spatial databases. [4]
  - Describe two features of distributed databases. [2]
  - Identify and compare a NoSQL database with a relational database. [4]
  - Explain the role of online analytical processing (OLAP) in business intelligence. [3]
  - Explain the COMMIT transaction control language (TCL) command. [1]
  - Other than COMMIT, identify another transaction control language (TCL) command. [1]

